

Project: Shop Desk

A spec-driven build — requirements only, two hours

This is a specification, not a tutorial. There are no prompts to copy. You will write a spec, hand it to your coding agent — Claude Code or OpenCode — and answer for everything it produces.

The brief. The Shop Desk answers customers for a small business: what's in stock, what it costs, and can I order three of them. It works from a catalogue file, builds a typed order when the customer confirms, and hands the genuinely stuck conversations to a human.

The theme of this project is **not wasting money**. A price question must not cost you a reasoning model. The process default is the cheap model and only the agent that needs more overrides it. The escalation handoff transfers a trimmed conversation, not the entire log. And no price ever reaches a customer that is not in your catalogue. Every requirement below is shaped by that.

Your first project routed conversations; your second ran reviewers in parallel. This one is about what each turn costs and how a long conversation stays affordable.

The rules

1. **Specification before implementation.** No source file until Phase 0 is committed. Your git history is the evidence and it is graded.
2. **The agent writes the code. You own it.** Anything you cannot explain, you have not finished.
3. **Requirements are numbered and testable.** "FR-3 works" means you can show it on demand.
4. **The clock is real.** Falling behind means cutting from the bottom of the cut list.

The clock

Time	Phase	What exists at the end of it
0:00–0:35	Phase 0 — specify	Four spec artifacts, committed, no code
0:35–1:05	Phase 1 — catalogue and the fast path	FR-1 ... FR-4
1:05–1:40	Phase 2 — orders, limits and truth	FR-5 ... FR-9
1:40–1:55	Phase 3 — escalation and interface	FR-10 ... FR-13
1:55–2:00	Demo	One long conversation, one order, one escalation

Cut list, in this order: FR-11, then FR-9, then the agent-level hooks inside FR-11. Never cut FR-3 or FR-6 — the fast path and the catalogue guardrail are what this project is for.

Phase 0 — the specification gate

Four documents before any implementation. Spec Kit produces this shape; plain markdown is acceptable as long as all four exist and agree.

constitution.md — the rules the build may not break. At minimum: which model is the process default and which agents may override it; that a quoted price must come from the catalogue; that secrets live only in **.env**; that no tool raises into the runner.

spec.md — what the Desk does as behaviour, every requirement below in your own words, and the three things it deliberately will not do.

plan.md — the architecture: which agents exist, which model each runs on and why, what every tool takes and returns, and the shape of each structure crossing a boundary.

tasks.md — ordered, independently verifiable tasks, each naming the requirement it serves.

Gate: all four committed before the first line of code. One commit holding both a spec and an implementation fails the phase even if the code runs.

Phase 1 — catalogue and the fast path

FR-1 — One catalogue, three levels of model configuration

Products live in **catalogue.json** and are reachable only through tools. The process runs a cheap model by default, set **globally**; the agent that has to reason about an order overrides it on **itself**; one re-quote path overrides it for a **single run**.

```
{
  "shop": "Al-Noor Electronics",
  "currency": "PKR",
  "products": [
    {"sku": "KTL-01", "name": "Electric kettle 1.7L", "price": 4200, "stock": 12},
    {"sku": "FAN-22", "name": "Pedestal fan", "price": 9800, "stock": 0}
  ]
}
```

Done when: you can point at all three overrides in the code, say which model answers a plain question, and delete the global default to watch exactly one behaviour change.

FR-2 — The customer lives in context

A **ShopContext** is passed to every run and read by tools. It never appears in prompt text.

```
from dataclasses import dataclass

@dataclass
class ShopContext:
    shop: str
    currency: str
    customer_id: str
    tier: str = "walk_in"      # "walk_in" or "regular"
```

Done when: a tool reads **tier** through the wrapper, that tool's generated schema has no wrapper parameter, and grepping your prompts finds no customer id.

FR-3 — The fast path: a price answer that costs one model call

"What does the kettle cost?" must be answered by the lookup tool's own output, with the model never seeing the result and never writing a sentence around it. Ordinary questions keep the normal loop.

Done when: the trace for a price question shows one model call and the trace for an order question shows more; you can state how many calls each path costs; and you can name one thing the Desk loses by answering this way.

FR-4 — Instructions that know the hour

The system prompt is built per turn from context and the clock: outside shop hours the Desk stops promising same-day delivery and says when it opens.

Done when: the same question at two different simulated hours produces two different promises, and you can print the resolved prompt before any model call.

Phase 2 — orders, limits and truth

FR-5 — A confirmed order is a typed object

When the customer confirms, the Desk's output is a structure, not prose.

```
from typing import Literal
from pydantic import BaseModel

class LineItem(BaseModel):
    sku: str
    qty: int
    unit_price: float

class Order(BaseModel):
    order_id: str
    status: Literal["draft", "confirmed", "escalated"]
    items: list[LineItem]
    total: float
```

Done when: the total is recomputed in Python from the line items rather than trusted from the model, and a mismatch between the two is reported rather than silently accepted.

FR-6 — No invented prices: an output guardrail against the catalogue

An output guardrail reads the finished answer and refuses it if it quotes a price or a SKU that is not in `catalogue.json`. This is a data check, not a vibe check — it compares against the file.

Done when: editing a price in the catalogue makes a previously passing answer fail, an out-of-stock item is never sold, and the refusal is caught and turned into a polite message rather than a traceback.

FR-7 — Tools that are off, and tools that are earned

Two different controls, both present: one seasonal tool switched **off statically** so nobody sees it, and one tool available only to customers whose tier is `regular`.

Done when: the same question asked as `walk_in` and as `regular` offers the model different tools, the seasonal tool appears in no schema at all, and you can explain why "off" and "not offered" are the same thing to the model.

FR-8 — A pricing specialist that returns a number, and a turn budget

A Pricing specialist is exposed as a tool to the Desk and answers with a figure, not a paragraph. Every conversation runs under a turn ceiling that raises, is caught, and ends the conversation politely.

Done when: the Desk's reply contains the specialist's number in the Desk's own wording, and you can state the ceiling you chose and what a customer sees when it is hit.

FR-9 — Specialists cloned from one base

The Pricing and Escalation agents are clones of a single base agent, differing only in instructions and model settings.

Done when: you can show with `is` comparisons what the clones share with the base and what they do not, and neither clone restates the model it inherits.

Phase 3 — escalation, lifecycle and interface

FR-10 — Escalation carries a typed reason and a trimmed history

When the Desk gives up, it hands the conversation to a Human Escalation agent. The handoff states **why** as a typed value, and the history that travels with it is filtered — the specialist should not inherit every tool call the Desk made.

The SDK ships filters for this (`agents.extensions.handoff_filters`). Use one or write your own; either way your `spec.md` must say what was removed and what the specialist still needs to be useful.

Done when: the escalation agent answers sensibly without the tool noise, the reason arrives as a typed value rather than a sentence, and you can show the before and after of the transferred history.

FR-11 — What every turn cost

Run-level hooks add a per-conversation cost line: turns, tokens and which model served each turn. Agent-level hooks are attached to the Pricing specialist only. A custom runner records each run's model and token count.

Done when: the cost line distinguishes fast-path turns from reasoning turns, and the numbers come from the run context rather than an estimate.

FR-12 — A long conversation in a browser

A Chainlit interface holds a conversation of at least ten turns. History is carried per session and trimmed past a turn count you choose and justify.

Done when: turn eleven still remembers the order under discussion, two browser windows do not share a basket, and you can say what the trimming rule throws away first.

FR-13 — One conversation, one trace

Tracing is enabled under your own key. A whole customer conversation is one trace.

Done when: you can open it, point to the fast-path turns and the reasoning turns, and name the most expensive turn in the conversation.

Non-functional requirements

- **NFR-1 — Secrets.** Keys in `.env`, gitignored, and never printed. A missing key fails at startup with a sentence, not a stack trace.
- **NFR-2 — Cost.** Every agent declares its own settings. The cheap model is the default and every override is justified in `plan.md`.
- **NFR-3 — Truth.** No price, stock figure or SKU reaches a customer unless it came from the catalogue this run.
- **NFR-4 — Failure.** A tool meeting bad data returns a sentence the model can use. A tool that raises into the runner is a defect.
- **NFR-5 — Provenance.** `git log` shows the four Phase 0 artifacts before the first code commit.

Concept coverage

Part	Forced by
0–2 Setup, keys, Gemini	FR-1, NFR-1
3 Runner and asyncio	FR-1, FR-12
4 Model configuration	FR-1 — global, agent and run levels together
5 Tools	FR-1, FR-2
6 Model settings	FR-8, FR-9, NFR-2
7 Local context	FR-2
8 Dynamic instructions	FR-4
9 Cloning	FR-9
10 Tracing	FR-13
11 Agents as tools	FR-8
12 Handoffs	FR-10 — typed reason and filtered history
13 Advanced tool control	FR-3 (fast path), FR-7, FR-8 (ceiling)
14 Structured output	FR-5
15 Guardrails	FR-6 (output, checked against data)
16 Lifecycle hooks	FR-11
17 Run lifecycle hooks	FR-11
18 Custom runners	FR-11
19 Chainlit	FR-12
20 Practice with an agent CLI	The whole project — you are driving one

Definition of done

Requirement	How it is checked
Spec preceded code	<code>git log</code> order, artifacts first
Three config levels present	Point at all three; delete the global one
Context never in the prompt	Tool schema has no wrapper; grep the prompts
Fast path costs one call	Two traces, two call counts
Order total recomputed	Planted mismatch is caught
Prices come from the file	Edit a price, watch an answer fail
Tools differ by tier	Same question, two tiers, two tool sets
Escalation carries a typed reason	Reason is a value, not a sentence
History is trimmed on transfer	Before and after shown
Ten turns and still coherent	Turn eleven remembers the basket

Defending your work

1. Which model answered the last question, and which of the three configuration levels decided that?
2. A price question costs one model call. What did you give up to get that?
3. Your guardrail compares against the catalogue. What happens if the catalogue is edited mid-conversation?
4. What exactly did your handoff filter remove, and how do you know the specialist did not need it?
5. Why is the seasonal tool invisible rather than refused?
6. Where does the token count in your cost line come from?
7. Your trimming rule drops something first. What, and why is that the safest thing to lose?
8. Show the clones' shared and independent attributes with `is`. Which sharing would bite you later?

If you finish early

- Persist orders and let the Desk answer "what did I order last week?".
- Add an input guardrail that refuses bargaining and routes it to escalation instead.
- Give the fast path a cache so a repeated price question costs nothing at all.
- Put your own shop's real catalogue behind it and count how many answers the guardrail blocks.